

WHITE PAPER V2.0

Shokunin The AI Engineering Ecosystem

Declarative ecosystem management, persistent memory, multi-runtime orchestration, and self-healing infrastructure for AI-augmented development.

swagger

Version 2.0 · 2026.05

Shokunin Project

| Contents

01	Ecosystem Overview	03
02	Architecture & Components	04
03	Memory System	06
04	Skills Ecosystem	08
05	Update System	10
06	Operations & Recovery	13
07	Component Inventory	15

| Ecosystem Overview

Shokunin is an AI engineering ecosystem built on OpenCode. It provides persistent memory across sessions, 38 specialized skills, multi-runtime script orchestration, and a declarative self-update system.

The name **Shokunin** (職人) refers to the Japanese concept of a master craftsman — someone who refines their tools and process continuously. The ecosystem embodies this philosophy: every component is designed to be improved, every process is documented, and the system itself maintains itself.

Key Capabilities

- Persistent memory via ChromaDB vector database across AI sessions
- 38 specialized skills covering frontend, backend, mobile, infra, content, security
- Multi-runtime — Windows PowerShell, Linux Bash, Python, Node.js
- MCP protocol for tool standardization across clients
- Declarative update system — drift detection, safe updates, instant rollback
- 3 LLM providers — NVIDIA Kimi K2.6 (primary), OpenCode Go (fallback), Ollama (local)

Scope

Metric	Value
Total files	1,093
Core scripts	17 (13 PowerShell + 4 Bash)
Python services	2 (chroma-helper.py + mcp-server.py)
Skills	38
LLM providers	3
MCP servers	3 (memory, filesystem, fetch)
Managed components	24
Protected data paths	4

| Architecture & Components

The ecosystem is organized in layers: config → orchestration → memory → skills. Each layer has a clear boundary and defined interfaces.

Layer Architecture

Layer	Path	Responsibility
Config	<code>~/.config/opencode/</code>	Provider setup, MCP servers, agent definitions, skill discovery
Instructions	<code>~/AGENTS.md</code> , <code>~/claude/CLAUDE.md</code>	Global rules for AI behavior, memory system instructions
Orchestration	<code>~/shokunin/scripts/</code>	Session wrapping, memory persistence, healthchecks, cleanup
Memory	<code>~/shokunin/memory/</code>	ChromaDB vector DB, MCP server, session files
Skills	<code>~/.config/opencode/skills/</code>	38 specialized SKILL.md files with scripts and references
Agent Skills	<code>~/agents/skills/</code>	18 additional skills (playwright, code-review, kami, etc.)
Update System	<code>~/shokunin/</code>	Declarative manifest, drift detection, templates, backups

Request Flow

A typical user session follows this path:

- `run-opencode.ps1` wraps the session with session ID, environment variables, and checkpoint timer
- OpenCode loads config from `opencode.json` including MCP servers and agents
- Memory MCP server (`mcp-server.py`) provides `store_context` , `search_context` , `get_session_summary`
- Skills are discovered from `skills/` directories and loaded on demand
- On session end, the wrapper saves context to ChromaDB and markdown fallback

MCP Servers

Server	Provider	Function
memory	mcp-server.py	ChromaDB vector memory via JSON-RPC
filesystem	@modelcontextprotocol/server-filesystem	Path-validated file access for subagents
fetch	@modelcontextprotocol/server-fetch	Web content retrieval, API calls

The memory MCP server is the only custom server. Filesystem and fetch use the standard MCP implementations. The memory server shares the same ChromaDB database as `chroma-helper.py` — they are two interfaces to the same data store.

Memory System

Persistent ChromaDB vector memory that survives across AI sessions. Every conversation, decision, and command is stored and searchable.

Dual-Interface Design

The memory system has two interfaces sharing the same ChromaDB database:

Interface	Protocol	Use Case
<code>chroma-helper.py</code>	CLI (Python script)	Fast batch operations, scripting, fallback
<code>mcp-server.py</code>	JSON-RPC (stdin/stdout)	From AI agents via MCP tool calls

Entry Types

Each memory entry is tagged with a type, project, and custom tags for filtering:

Type	When	Example
<code>decision</code>	Architecture choice made	"Use Prisma over Drizzle for relational data"
<code>file</code>	File created or modified	"Created auth middleware in src/middleware/auth.ts"
<code>command</code>	Notable command executed	"npm run build completed with 0 errors"
<code>checkpoint</code>	Periodic state snapshot	Auto-saved every 5 minutes during session
<code>session_end</code>	Session complete	Full summary with decisions, files, next steps
<code>general</code>	Everything else	Notes, observations, context

Fallback Chain

The memory system degrades gracefully:

- 1. Primary: ChromaDB vector database
- 2. Backup: Markdown files in `memory/sessions/`
- 3. Emergency: Session logs in `memory/sessions/*.log`

The `run-opencode.ps1` wrapper tries ChromaDB first, falls back to markdown files, and logs raw console output regardless of success.

| Skills Ecosystem

38 skills organized across 8 domains. Each skill is a `SKILL.md` file with optional scripts, references, and assets.

Skill Domains

Domain	Count	Skills
Infrastructure	5	docker, kubernetes, terraform, ci-cd, db-admin
Backend	4	api-forge, auth-architect, db-sculptor, error-handler
Frontend	6	component-forge, responsive-engine, motion-craft, landing-craft, ui-ux-pro-max, aesthetic-web
Mobile	2	flutter, react-native
Quality	3	test-commander, performance-profiler, code-review
Content	6	communication, content-marketing, business-proposals, seo-geo, translate-craft, documentation
Productivity	8	git-workflow, windows-powershell, runbook-gen, strategy, design, finance, legal-counsel, whendone-plus
Documents	4	kami, portfolio-auto, shokunin-update, humanize

Skill Structure

Each skill follows a standard structure validated by `validate-skills.ps1` :

- YAML frontmatter with name, description, metadata
- Workflow section with step-by-step instructions
- Error handling section with common failure modes
- Sources section for attribution

Discovery

OpenCode discovers skills from `~/.config/opencode/skills/` , `~/.agents/skills/` , and `~/.claude/skills/` . They are listed in the `skill` tool description and loaded on demand.

| Update System

A declarative approach to ecosystem maintenance. One manifest describes every component. Drift detection compares manifest vs reality. Apply synchronizes them with automatic backup.

The Problem

Without a single source of truth, updating the ecosystem required manual coordination across 5+ directories. Changing a script path meant finding and editing references in `opencode.json` , `CLAUDE.md` , `AGENTS.md` , `run-opencode.ps1` , and `mcp-server.py` . Mistakes were inevitable.

The Solution

A **declarative manifest** (`~/.shokunin/shokunin.json`) that defines every component: path, type, runtime, template, and protection rules. Variables avoid hardcoded paths. Templates make config files regeneratable.

Four-Phase Workflow

1. status — Detect Drift

Computes SHA-256 hashes of every static file and compares against the manifest. Reports OK, MISSING, and PROTECTED counts.

```
PS> & "$env:USERPROFILE\.shokunin\scripts\shokunin-update.ps1" status

Components: 24 total
  20 OK
  3 PROTECTED (data - never modified)
```

2. plan — Preview Changes

Read-only. Shows exactly which files would be created, modified, or left alone.

```
PS> & "$env:USERPROFILE\.shokunin\scripts\shokunin-update.ps1" plan

Plan:
MODIFY 2 files
  opencode.json: ~/.config/opencode/opencode.json
  AGENTS.md: ~/AGENTS.md
Backup: 2 files
Data: 3 paths excluded
```


3. apply — Execute with Backup

Backs up each file to `~/.shokunin/backups/<timestamp>/` , applies changes, and runs verification automatically.

```
PS> & "$env:USERPROFILE\.shokunin\scripts\shokunin-update.ps1" apply -Confirm
```

4. rollback — Instant Recovery

Restores files from a specific backup timestamp. Lists available backups if no timestamp provided.

```
PS> & "$env:USERPROFILE\.shokunin\scripts\shokunin-update.ps1" rollback -Timestamp  
20260514-120000
```

Data Protection

Four paths are marked `NEVER_MODIFY` in the manifest:

- `~/.shokunin/memory/chroma_db/` — vector database
- `~/.shokunin/memory/sessions/` — session logs
- `~/.shokunin/logs/` — healthcheck logs
- `~/.shokunin/backups/` — backup archives

The engine checks the protection rule before every operation. These paths cannot be modified even with `apply -Confirm` .

| Operations & Recovery

Daily maintenance procedures and emergency recovery steps.

Weekly Maintenance

```
# 1. Check ecosystem drift
& "$env:USERPROFILE\.shokunin\scripts\shokunin-update.ps1" status

# 2. Full healthcheck
& "$env:USERPROFILE\.shokunin\scripts\memory-healthcheck.ps1"
```

The weekly healthcheck also creates a compressed backup of the memory directory and reports disk space.

After Configuration Changes

When changing paths, models, or providers:

1. Edit variables in `shokunin.json`
2. Run `plan` to verify which files will change
3. Run `apply -Confirm` to regenerate templates

Recovery Procedures

Rollback After Failed Update

```
# List available backups
& "$env:USERPROFILE\.shokunin\scripts\shokunin-update.ps1" rollback

# Restore from specific timestamp
& "$env:USERPROFILE\.shokunin\scripts\shokunin-update.ps1" rollback -Timestamp 20260514-120000
```

Manual Recovery

If the update script itself is broken, backups are plain files with `.bak` extension:

```
# Restore manually
Copy-Item "$env:USERPROFILE\.shokunin\backups\20260514-120000\opencode.json.bak" `
"$env:USERPROFILE\.config\opencode\opencode.json"
```

Full Ecosystem Reset

To regenerate all template-based files from scratch:

```
# This only affects template-based files (configs, instructions)
# Protected data (chroma_db, sessions) is never touched
& "$env:USERPROFILE\.shokunin\scripts\shokunin-update.ps1" apply -Confirm
```

AI Session Integration

When working inside an AI session with the `shokunin-update` skill active:

1. I read the manifest and check status
2. I present drift findings to you
3. I run plan/apply with your confirmation
4. Changes are saved to ChromaDB memory with context
5. Healthcheck runs automatically after apply
6. On failure, I suggest rollback

The update system was tested against all 25 healthchecks. Every fix was verified. The system maintains itself — it's included in the manifest as a managed component.

Component Inventory

Complete inventory of all 24 managed components in the ecosystem.

Core Scripts

File	Runtime	Type	Template
chroma-helper.py	Python	Static	N/A
mcp-server.py	Python	Template	mcp-server.py.tpl
run-opencode.ps1	PowerShell	Template	run-opencode.ps1.tpl
save-memory.ps1	PowerShell	Template	save-memory.ps1.tpl
search-memory.ps1	PowerShell	Template	search-memory.ps1.tpl
memory-healthcheck.ps1	PowerShell	Template	memory-healthcheck.ps1.tpl
test-memory.ps1	PowerShell	Template	test-memory.ps1.tpl
shokunin-update.ps1	PowerShell	Static	N/A
read-transcript.ps1	PowerShell	Static	N/A
seed-memory.ps1	PowerShell	Static	N/A
scan-cleanup.ps1	PowerShell	Static	N/A
validate-skills.ps1	PowerShell	Static	N/A
weekly-healthcheck.ps1	PowerShell	Static	N/A

Linux Scripts

File	Runtime	Type
run-opencode.sh	Bash	Template
memory-healthcheck.sh	Bash	Static
profile.sh	Bash	Static
weekly-maintenance.sh	Bash	Static

Config Templates

Template	Generates	Variables
opencode.json.tpl	~/.config/opencode/opencode.json	MODEL, MCP_MEMORY_PATH, FILESYSTEM_ROOT
AGENTS.md.tpl	~/AGENTS.md	SHOKUNIN_DIR
CLAUDE.md.tpl	~/.claude/CLAUDE.md	SHOKUNIN_DIR

Protected Paths

Path	Content
~/.shokunin/memory/chroma_db/	Vector database with all session memory
~/.shokunin/memory/sessions/	Markdown session logs and checkpoints
~/.shokunin/logs/	Healthcheck and cleanup operational logs
~/.shokunin/backups/	Automatic pre-update backups